

Introducción a los Sistemas Distribuidos

Redes y Aplicaciones Distribuidas

<75.43> <95.60> <TA048>

TP N°1 File Transfer

Grupo N°22

Fernandez Paz, Alejo - 112142

Tissera, José Evaristo - 112788

Vandevallé Miranda, Camila - 112777

Calandria, Alén - 112103

Arias Santiago - 112194

Índice

Introducción.....	3
Hipótesis y suposiciones realizadas	3
1.1. Desarrollo	3
2. Implementación.....	4
2.1. Server-client.....	4
3. Pruebas.....	4
3.1. Configuración del entorno	5
3.2. Archivos Utilizados	5
3.3. Metodología.....	5
3.4. Resultados obtenidos.....	6
3.5. Ejecuciones de prueba.....	7
3.6. Observaciones.....	8
4. Preguntas a responder	9
4.1. Describa la arquitectura Cliente-Servidor.....	9
4.2. ¿Cuál es la función de un protocolo de capa de aplicación?	9
4.3. Detalle el protocolo de aplicación desarrollado en este trabajo.	9
4.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?	10
4.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.	10
4.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?.....	11
5. Dificultades encontradas.....	11
Pérdida del último ACK	11
Manejo de retransmisiones en el handshake.....	12
Go-Back-N más lento de lo esperado	12
Timeout dinámico.....	12
6. Conclusión	13

Introducción

El presente trabajo práctico tiene como objetivo el desarrollo de una aplicación de transferencia de archivos sobre el protocolo UDP, implementando mecanismos de confiabilidad a nivel de aplicación. Dado que UDP no garantiza la entrega de paquetes ni el orden de los mismos, se implementaron dos protocolos de recuperación de errores: Stop & Wait y Go-Back-N.

La aplicación permite realizar operaciones de subida y descarga de archivos entre un cliente y un servidor, utilizando sockets UDP. Para garantizar la correcta transferencia de datos, se incorporaron mecanismos de control de secuencia, confirmaciones (ACKs), retransmisiones ante pérdidas y control de timeouts.

Además, se realizaron pruebas en un entorno simulado mediante Mininet, evaluando el comportamiento de ambos protocolos bajo diferentes condiciones de red, incluyendo escenarios con pérdida de paquetes.

Hipótesis y suposiciones realizadas

1.1. Desarrollo

Para el desarrollo de la aplicación se asumió que:

- La comunicación se realiza sobre UDP, por lo que no existe garantía de entrega, orden ni duplicación de paquetes.
- El tamaño máximo de los paquetes está limitado por una constante predefinida (MAX_PACKET_SIZE).
- El cliente y el servidor comparten el mismo formato de paquetes y protocolo de comunicación.
- Se utiliza un mecanismo de numeración de secuencia para identificar cada paquete de datos.
- Se emplean timeouts para detectar la pérdida de paquetes o ACKs.
- En Go-Back-N, el receptor solo acepta paquetes en orden y descarta aquellos fuera de secuencia.

- La red puede presentar pérdida de paquetes (simulada con Mininet), lo que obliga a retransmisiones.

Estas suposiciones permiten modelar un entorno realista donde es necesario implementar confiabilidad sobre un protocolo no confiable como UDP.

2. Implementación

2.1. Server-client

La aplicación está compuesta por un cliente y un servidor que se comunican a través de sockets UDP. Para gestionar la concurrencia, el servidor atiende las solicitudes de múltiples clientes mediante threads, adoptando un esquema similar al de TCP: cada cliente es derivado a un nuevo puerto, lo que permite manejar transferencias simultáneas de forma mucho mas sencilla.

Toda operación de transferencia, ya sea de subida o descarga, comienza con un handshake inicial en el que el cliente envía una solicitud indicando su intención. Dado que UDP no contempla el concepto de conexión, el servidor no puede conocer el estado del canal de comunicación de antemano por lo tanto, debe ser capaz de interpretar correctamente los paquetes recibidos para luego responder al cliente con el nuevo puerto en el que se llevará a cabo la transferencia.

Para garantizar la confiabilidad en las transferencias, se implementaron dos protocolos de recuperación de errores:

- Stop & Wait: El emisor envía un paquete y espera su confirmación antes de enviar el siguiente. En caso de timeout, el paquete es retransmitido.
- Go-Back-N: El emisor puede enviar múltiples paquetes sin esperar confirmación, manteniendo una ventana de transmisión. El receptor envía ACKs acumulativos y descarta paquetes fuera de orden. Ante la pérdida de un paquete, el emisor retransmite todos los paquetes a partir del último no confirmado.

Para garantizar la robustez del sistema, se implementaron mecanismos adicionales como:

- Reenvío del último ACK en caso de recibir paquetes duplicados.
- Manejo de pérdida del ACK final mediante un período de espera adicional (linger).
- Control de sesiones activas en el servidor para evitar la creación de múltiples transferencias ante retransmisiones del cliente.

Estos mecanismos permiten asegurar la correcta transferencia de archivos incluso en presencia de pérdida de paquetes.

3. Pruebas

Para evaluar el correcto funcionamiento de la aplicación, se realizaron distintas pruebas utilizando el emulador de redes Mininet. Este entorno permitió simular condiciones de red controladas, incluyendo pérdida de paquetes.

Se llevaron a cabo pruebas de transferencia de archivos entre dos hosts (h1 como servidor y h2 como cliente), utilizando ambos protocolos implementados: Stop & Wait y Go-Back-N y también se realizaron pruebas con múltiples host utilizando distintos protocolos y acciones.

3.1. Configuración del entorno

Para la topología simple, se crearon dos hosts conectados a un switch:

```
sudo mn --topo single,2 --link tc,loss=X
```

donde X representa el porcentaje de pérdida de paquetes.

El servidor se ejecutó en h1 y el cliente en h2, utilizando direcciones IP del rango 10.0.0.0/8.

Para las pruebas de concurrencia, se extendió la topología a cinco hosts mediante xterm, abriendo una consola independiente por cada uno. En este escenario, h1 actuó como servidor mientras que h2, h3, h4 y h5 operaron como clientes simultáneos, realizando operaciones de carga y descarga de imágenes de distintos tamaños y con distintos protocolos, permitiendo así evaluar el comportamiento del servidor ante múltiples transferencias concurrentes.

3.2. Archivos Utilizados

Se realizaron pruebas con archivos de distintos tamaños para analizar el comportamiento de los protocolos:

- 100 KB
- 1 MB
- 5 MB
- 10MB

3.3. Metodología

Para cada combinación de protocolo, tamaño de archivo y porcentaje de pérdida, se midió el tiempo total de transferencia utilizando el comando time. A partir de estos valores, se calculó el throughput en Mbps.

Throughput = tamaño del archivo (bits) / tiempo (segundos)

1KB = 1024 x 8 b

1MB = 1024 x 1024 x 8 b

1 b/s = 1/10⁶ Mbps

Las pruebas se realizaron para:

- Stop & Wait (S&W)
- Go-Back-N (GBN)

bajo dos condiciones:

- Sin pérdida (0%)
- Con pérdida (15%)

3.4. Resultados obtenidos

Protocolo	Tamaño	Pérdida	Tiempo (s)	Throughput (Mbps)
GBN	100 KB	0%	0.058	14.12
S&W	100 KB	0%	0.062	13.22
GBN	1 MB	0%	0.132	63.55
S&W	1 MB	0%	0.171	49.06
GBN	5 MB	0%	0.406	103.30
S&W	5 MB	0%	0.526	79.73
GBN	10 MB	0%	0.719	116.67

S&W	10 MB	0%	0.922	90.99
GBN	100 KB	15%	4.226	0.19
S&W	100 KB	15%	18.520	0.04
GBN	1 MB	15%	9.441	0.89
S&W	1 MB	15%	37.554	0.22
GBN	5 MB	15%	15.774	2.66
S&W	5 MB	15%	49.512	0.85
GBN	10 MB	15%	29.985	2.80
S&W	10 MB	15%	53.702	1.56

3.5. Ejecuciones de prueba

Archivo 100KB, 0% loss, gbn. Upload

```
[INFO] Recibiendo "arbol.jpg" (105729 bytes) en puerto 50611 [protocol=2]
[INFO] [GBN] Recepción completada: 105729 bytes recibidos
[INFO] Upload "arbol.jpg" completado
```

```
[INFO] Starting upload of "/home/jtissera/Desktop/arbol.jpg" -> "arbol.jpg"
[INFO] [GBN] Transferencia completada: 105729 bytes enviados
[INFO] Upload de "arbol.jpg" completado
python3 upload -H 10.0.0.1 -p 9000 -s /home/jtissera/Desktop/arbol.jpg -n -r
0.04s user 0.01s system 85% cpu 0.057 total
```

Archivo 100KB, 0% loss, saw. Upload

```
[INFO] Recibiendo "arbol.jpg" (105729 bytes) en puerto 50130 [protocol=1]
[INFO] [S&W] Recepción completada: 105729 bytes recibidos
[INFO] Upload "arbol.jpg" completado
```

```
[INFO] Starting upload of "/home/jtissera/Desktop/arbol.jpg" -> "arbol.jpg"
[INFO] [S&W] Transferencia completada: 105729 bytes enviados
[INFO] Upload de "arbol.jpg" completado
python3 upload -H 10.0.0.1 -p 9000 -s /home/jtissera/Desktop/arbol.jpg -n -r
0.04s user 0.01s system 89% cpu 0.058 total
```

Archivo 10MB, 15% loss, gbn. Upload

```
[INFO] Recibiendo "planta.jpg" (9908939 bytes) en puerto 41330 [protocol=2]
[INFO] [GBN] Recepción completada: 9908939 bytes recibidos
[INFO] Upload "planta.jpg" completado
```

```
[INFO] Starting upload of "/home/jtissera/Desktop/planta.jpg" -> "planta.jpg"
[INFO] [GBN] Transferencia completada: 9908939 bytes enviados
[INFO] Upload de "planta.jpg" completado
python3 upload -H 10.0.0.1 -p 9000 -s /home/jtissera/Desktop/planta.jpg -n -r
0.67s user 0.69s system 4% cpu 33.710 total
```

Archivo 10MB, 15% loss, saw. Download

```
[INFO] Enviando "planta.jpg" (9908939 bytes) en puerto 35284 [proto=1]
[INFO] [S&W] Transferencia completada: 9908939 bytes enviados
[INFO] Download "planta.jpg" completado
```

```
[INFO] Starting download of "planta.jpg" -> "/home/jtissera/Downloads/"
[INFO] [S&W] Recepción completada: 9908939 bytes recibidos
[INFO] Download de "planta.jpg" completado
python3 download -H 10.0.0.1 -p 9000 -d /home/jtissera/Downloads/ -n -r saw 0
.68s user 0.98s system 2% cpu 1:06.09 total
```

3.6. Observaciones

En condiciones sin pérdida, ambos protocolos presentan tiempos similares para archivos pequeños.

Vale destacar que, durante el desarrollo, se esperaba que Go-Back-N superara notoriamente a Stop & Wait en términos de rendimiento. Sin embargo, los resultados iniciales mostraron lo contrario. Frente a esto, se introdujo una mejora: el tamaño de la ventana de transmisión pasó a ser adaptativo. Cuando la red presenta alta inestabilidad y una tasa de pérdida elevada, la ventana se reduce a la mitad, disminuyendo así la cantidad

de paquetes que deben ser reenviados ante un error. Dado que nuestra implementación descarta todos los paquetes que llegan fuera de orden y retransmite la ventana completa, en escenarios muy adversos Go-Back-N tiende a comportarse de manera similar a Stop & Wait, la ventana se achica progresivamente hasta alcanzar tamaño 1. No obstante, en la transferencia de archivos de mayor tamaño, Go-Back-N logra una mejora de rendimiento considerable respecto del otro protocolo.

En presencia de pérdida de paquetes del 15%, ambos protocolos experimentan un aumento en los tiempos de transferencia. Stop & Wait se ve especialmente penalizado en este escenario, ya que cada retransmisión implica detener por completo el envío de nuevos datos hasta recibir confirmación. Go-Back-N, en cambio, al mantener múltiples paquetes en tránsito de forma simultánea, logra sostener un mejor desempeño.

En todos los casos, se verificó que los archivos transferidos llegan correctamente al destino, confirmando la confiabilidad de la implementación.

4. Preguntas a responder

4.1. Describa la arquitectura Cliente-Servidor.

La arquitectura cliente servidor, utilizada en este trabajo, es un modelo de desarrollo compuesto por 2 entidades. Por un lado tenemos a los clientes que se encargan de interactuar con el usuario, presentar la información y solicitar servicios al servidor mediante peticiones, por otro lado tenemos al servidor que se encarga de gestionar todas las peticiones de los clientes concurrentemente, almacenar los datos, y ofrecer los servicios requeridos por los clientes. Esta comunicación es bidireccional y sucede a través de la red.

En este trabajo construimos una simple aplicación cliente-servidor que nos permita subir archivos a un servidor desde un cliente para que luego otro cliente, o el mismo, pueda descargarlos. En este trabajo se hizo principal foco en la comunicación entre el cliente y servidor, creando un protocolo de comunicación confiable en capa de aplicación utilizando UDP como protocolo de transporte.

4.2. ¿Cuál es la función de un protocolo de capa de aplicación?

La principal función de un protocolo de capa de aplicación es brindar acceso a los servicios de las demás capas a las aplicaciones, agregar servicios necesarios y definir los protocolos que utilizan estas para intercambiar datos.

Existen diversos protocolos de capa de aplicación como:

DHCP, que se encarga de realizar la configuración dinámica de un host, asignándole IP junto a otros parámetros de configuración de red.

HTTP, que define la comunicación y transferencia de información a través de archivos.

DNS, que se encarga de resolver nombres de dominio en direcciones IP.

SMTP, que es utilizado para el intercambio de mensajes de correo electrónico.

4.3. Detalle el protocolo de aplicación desarrollado en este trabajo.

En este trabajo desarrollamos un protocolo de aplicación que se encarga de brindar un servicio de transferencia de datos confiable sobre el protocolo de capa de transporte UDP, dado que UDP no garantiza la entrega de paquetes ni el orden de los mismos, se implementaron una serie de mecanismos a nivel de capa de aplicación para poder asegurar la confiabilidad en la transferencia de datos.

Se desarrollaron dos variantes del protocolo de aplicación, una que utiliza el protocolo Go-Back-N y otra que utiliza el protocolo Stop & Wait. El primero mas complejo en implementación pero mas eficiente, mientras que el segundo mas simple pero con problemas de rendimiento. En estos protocolos se utilizan mecanismos como sequence numbers, acknowledgment numbers, retransmisiones y manejo de timeouts.

4.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

La capa de transporte permite la comunicación entre aplicaciones mediante la multiplexación y demultiplexación.

El protocolo UDP ofrece las utilidades mínimas de multiplexación y demultiplexación y una verificación de errores mínima utilizando el checksum, este es no conectivo, no fiable, ligero y simple. Es utilizado cuando queremos priorizar la velocidad de transmisión por delante de la fiabilidad de los datos, se utiliza en aplicaciones en tiempo real que no pueden permitirse grandes retrasos, como streaming de video, VoIP y consultas DNS.

El protocolo TCP, en cambio, ofrece un servicio confiable orientado a la conexión, brinda el multiplexado y demultiplexado mínimo requerido como protocolo de capa de transporte pero a su vez brinda confiabilidad; asegura el orden; la entrega y la integridad de los datos, brinda control de flujo y control de congestión. Para proveer dichos servicios, este protocolo se vuelve muchos mas complejo y pesado que UDP por lo que se utiliza cuando la fiabilidad y la integridad de los datos son críticos, sacrificando la velocidad de transmisión. Por ejemplo se utiliza en la navegación web y correo electrónico.

4.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.

El protocolo desarrollado sí cuenta con mecanismos de control de congestión, presentes tanto en la variante Stop & Wait como en Go-Back-N. Si bien el enfoque principal de la implementación apunta a garantizar la confiabilidad de la transmisión detectando pérdidas, errores y reordenamientos para luego realizar las retransmisiones necesarias, es precisamente en la detección de pérdidas donde estos mecanismos de control entran en acción.

En ambos protocolos se implementan timeouts dinámicos: ante la necesidad de retransmitir un paquete, el tiempo de espera se duplica con cada intento consecutivo. Esta estrategia se basa en la suposición de que una causa probable de la pérdida es la congestión de la red, por lo que extender el intervalo de espera le otorga tiempo al sistema para que dicha congestión se alivie antes de reintentar el envío.

En el caso particular de Go-Back-N, se incorpora un mecanismo complementario de ventana adaptativa: ante la detección de pérdida de paquetes, el tamaño de la ventana de transmisión se reduce, disminuyendo la cantidad de datos en tránsito y aliviando así la presión sobre la red. A medida que la transmisión transcurre sin errores, la ventana se incrementa progresivamente.

4.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

Por lo visto en las pruebas y analizando el funcionamiento de los dos protocolos desarrollados podemos afirmar que en la mayoría de casos el protocolo Go-Back-N envía los archivos en menor cantidad de tiempo. Esto se debe a que este protocolo envía varios paquetes dentro de su ventana deslizante y espera la confirmación de estos para avanzar su ventana deslizante y poder mandar nuevos paquetes, mientras que el protocolo Stop & Wait solo envía un paquete y no envía ningún otro hasta recibir la confirmación de ese paquete, esto aumenta la simplicidad pero conlleva un aumento en la latencia.

La diferencia en rendimiento entre ambos protocolos se vuelve más notoria a medida que aumenta el tamaño del archivo y la tasa de pérdida de paquetes en la red. En Stop & Wait, cada pérdida implica esperar el vencimiento del timeout antes de retransmitir. Go-Back-N, en cambio, reacciona de forma más ágil, recibe un paquete fuera de orden y el receptor reenvía inmediatamente el último ACK enviado, desencadenando una retransmisión rápida de los paquetes pendientes sin necesidad de agotar el timeout.

La ventana adaptativa descrita anteriormente también cumple un rol clave, ante pérdidas reiteradas, la ventana se reduce progresivamente a la mitad, limitando la cantidad de paquetes que deben retransmitirse en cada intento. En el peor de los casos la ventana

puede achicarse hasta tamaño 1, equiparando el comportamiento de Go-Back-N al de Stop & Wait.

5. Dificultades encontradas

Durante el desarrollo del trabajo se presentaron diversas dificultades relacionadas con la implementación de confiabilidad sobre UDP y el manejo de condiciones adversas de red.

Pérdida del último ACK

En algunos casos, la transferencia fallaba al finalizar debido a la pérdida del último ACK enviado por el receptor. Esto provocaba que el emisor continuara retransmitiendo el último paquete, mientras que el receptor ya había finalizado la transferencia.

Para solucionar este problema, se implementó un período adicional de espera (linger) en el receptor, durante el cual se reenvía el último ACK en caso de recibir paquetes duplicados.

Manejo de retransmisiones en el handshake

Bajo condiciones de pérdida, el cliente podía reenviar la solicitud inicial de transferencia (UPLOAD_REQ) al no recibir respuesta del servidor. Esto provocaba que el servidor interpretara cada solicitud como una nueva conexión, generando múltiples transferencias simultáneas del mismo archivo.

Este problema se resolvió incorporando un mecanismo de deduplicación en el servidor, que permite identificar solicitudes repetidas y reutilizar la misma sesión de transferencia en lugar de crear nuevas.

Go-Back-N más lento de lo esperado

En su versión inicial, la implementación de Go-Back-N funcionaba correctamente y se comportaba según lo esperado en escenarios con baja pérdida de paquetes y archivos de tamaño reducido. Sin embargo, al incrementar la pérdida de paquetes, el protocolo comenzaba a mostrar un rendimiento inferior al anticipado, volviéndose más lento que Stop & Wait en lugar de superarlo.

La causa radicaba en la estrategia de recepción, ante la llegada de paquetes fuera de orden, estos eran descartados y se retransmitía la ventana completa. Esto generaba que mientras Stop & Wait solo debía reenviar un único paquete por cada pérdida, Go-Back-N debía retransmitir todos los paquetes de la ventana, multiplicando el tráfico innecesario y generando un efecto peyorativo en el rendimiento a medida que la red se volvía más inestable.

La solución adoptada fue incorporar una ventana dinámica que reduce su tamaño ante condiciones de red adversas, limitando así la cantidad de paquetes involucrados en cada retransmisión.

Timeout dinámico

Inicialmente se optó por trabajar con un timeout estático, decisión ocasionó serios problemas de rendimiento. Al no adaptarse al estado real de la red, este esquema presentaba dos comportamientos problemáticos según el valor elegido, si el timeout era demasiado corto, se generaban retransmisiones innecesarias, y si era demasiado largo, el sistema perdía tiempo esperando confirmaciones que no llegarían.

La solución fue reemplazarlo por un timeout dinámico, calculado mediante el mismo algoritmo utilizado por TCP, el algoritmo de Jacobson/Karels. Este método estima el tiempo de ida y vuelta de los paquetes (RTT) y ajusta el timeout en función de su variabilidad ($DevRtt$), logrando una adaptación continua y precisa a las condiciones cambiantes de la red.

6. Conclusión

Se logró implementar una aplicación de transferencia de archivos confiable sobre UDP, incorporando mecanismos de control de errores mediante los protocolos Stop & Wait y Go-Back-N.

A través de las pruebas realizadas, se pudo comprobar que ambos protocolos permiten garantizar la correcta entrega de los datos incluso en presencia de pérdida de paquetes. Sin embargo, se observaron diferencias significativas en su rendimiento.

En particular, Go-Back-N demostró ser más eficiente que Stop & Wait, especialmente en transferencias de mayor tamaño. Esto se debe a su capacidad de mantener múltiples paquetes en tránsito simultáneamente, reduciendo los tiempos de espera y mejorando el aprovechamiento del canal.

Por otro lado, Stop & Wait, si bien es más simple de implementar, presenta un rendimiento limitado debido a su naturaleza secuencial, lo que lo hace más sensible a la latencia y a la pérdida de paquetes.

El uso de herramientas como Mininet permitió validar el comportamiento del sistema en entornos controlados, simulando condiciones reales de red y evidenciando la importancia de los mecanismos de retransmisión.

En conclusión, se logró cumplir con los objetivos del trabajo, desarrollando una solución robusta que garantiza la transferencia confiable de archivos sobre un protocolo no confiable como UDP, y permitiendo comparar el desempeño de distintos enfoques de recuperación de errores.